

Database:

A **database** is an organized collection of data. There are many different strategies for organizing data to facilitate easy access and manipulation. A **database management system (DBMS)** provides mechanisms for **storing, organizing, retrieving and modifying data** form any users. **Database management systems allow for the access and storage of data without concern for the internal representation of data.**

Java programs communicate with databases and manipulate their data using the **Java Database Connectivity (JDBC) API**. A **JDBC driver** enables Java applications to connect to a database in a particular DBMS and allows us to manipulate that database using the JDBC API.

The **JDBC API** is a Java API that can access any kind of tabular data, especially data stored in a Relational Database. **JDBC** helps to write Java applications that manage these three programming activities:

- Connect to a data source, like a database
- Send queries and update statements to the database
- Retrieve and process the results received from the database in answer to our query

JDBC includes four components:

- ❖ **The JDBC API:** The JDBC API provides programmatic access to relational data from the Java programming language. Using the JDBC API, applications can execute SQL statements, retrieve results, and propagate changes back to an underlying data source. The JDBC API can also interact with multiple data sources in a distributed, heterogeneous environment. The JDBC API is part of the Java platform, which includes the Java Standard Edition (Java SE) and the Java Enterprise Edition (Java EE). The **JDBC 4.0 API** is divided into **two packages: java.sql and javax.sql**. Both packages are included in the Java SE and Java EE platforms.
- ❖ **JDBC Driver Manager:** The JDBC DriverManager class defines objects which can connect Java applications to a JDBC driver. DriverManager has traditionally been the backbone of the JDBC architecture. It is quite small and simple.
- ❖ **JDBC Test Suite:** The JDBC driver test suite helps us to determine that JDBC drivers will run our program. These tests are not comprehensive or exhaustive, but they do exercise many of the important features in the JDBC API.
- ❖ **JDBC-ODBC Bridge:** The Java Software Bridge provides JDBC access via ODBC drivers. JDBC bridge is used to access ODBC drivers installed on each client machine. Using ODBC requires configuring on our system a Data Source Name (DSN) that represents the target database. When Java first came out, this was a useful driver because most databases only supported ODBC access but now this type of driver is recommended only for experimental use or when no other alternative is available.

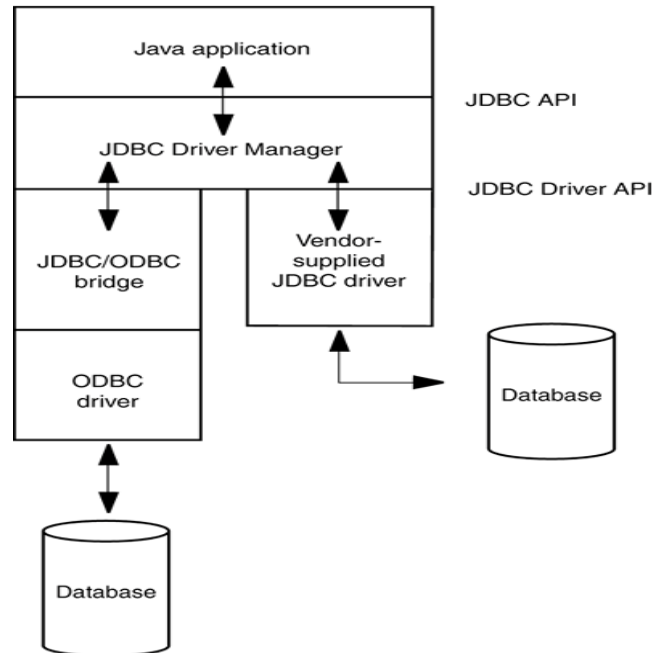


Fig: JDBC-to-database communication path

JDBC Driver Types

JDBC drivers are classified into the following types:

- ❖ A **type 1 driver** also known as **JDBC-ODBC Bridge Driver** translates JDBC to ODBC and relies on an ODBC driver to communicate with the database. Sun includes one such driver, the JDBC/ODBC bridge, with the JDK. However, the bridge requires deployment and proper configuration of an ODBC driver. When JDBC was first released, the bridge was handy for testing, but it was never intended for production use. For practical use there are better drivers than JDBC-ODBC drivers. Now a days in most cases it is only being used for the educational purposes.

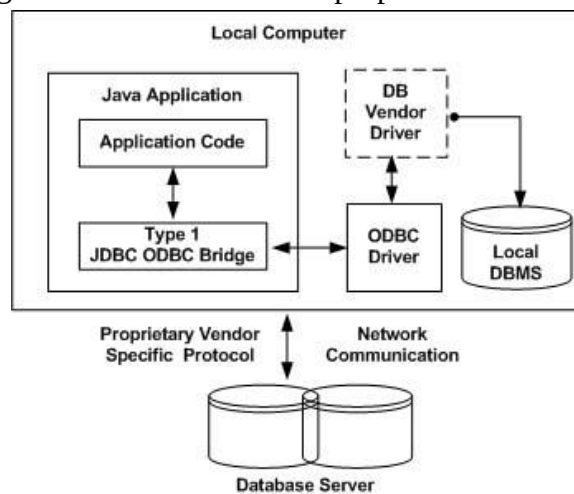
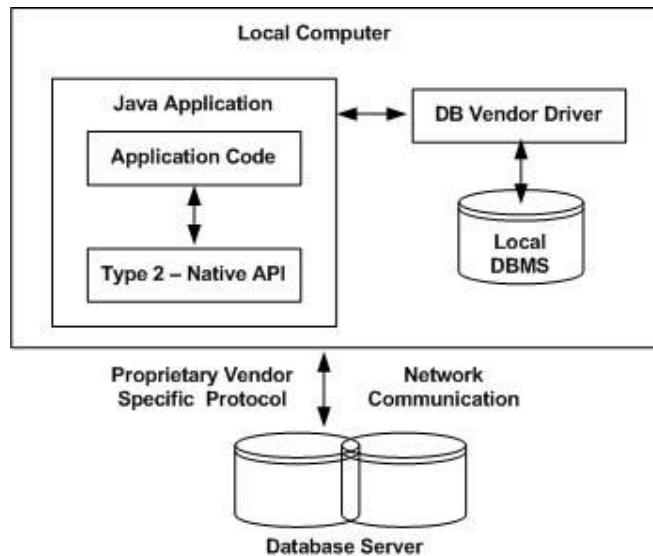


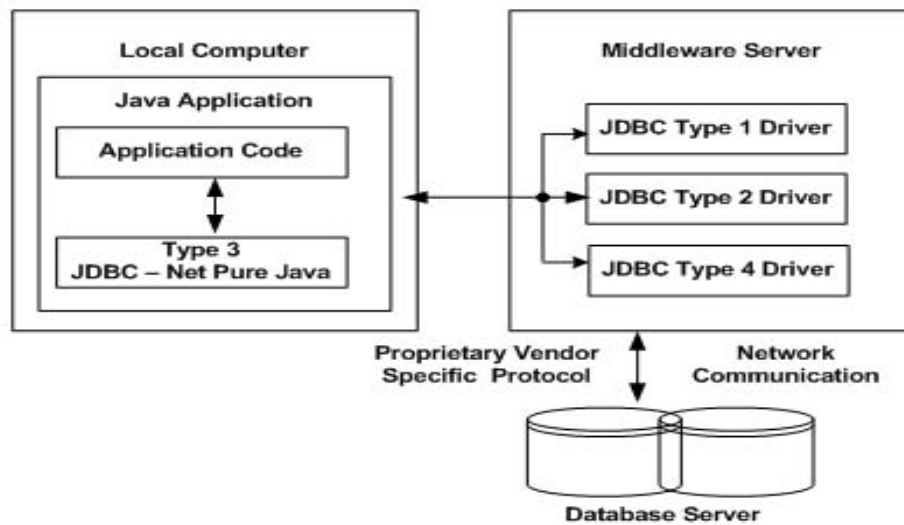
Fig: The JDBC-ODBC bridge

- ❖ A **type 2 driver also known as JDBC-Native API** is written partly in Java and partly in native code; it communicates with the client API of a database. When we use such a driver, we must install some platform-specific code in addition to a Java library. These drivers typically provided by the database vendors and used in the same manner as the JDBC-ODBC Bridge, the vendor-specific driver must be installed on each client machine. If we change the Database we have to change the native API as it is specific to a database and they are mostly obsolete now but we may increase some speed with a Type 2 driver, because it eliminates ODBC's overhead.



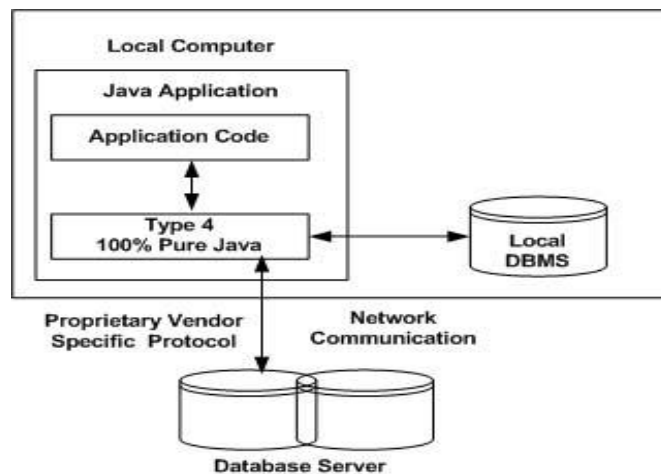
The Oracle Call Interface (OCI) driver is an example of a Type 2 driver.

- ❖ A **type 3 driver is also known as JDBC-Net pure Java**, is a pure Java client library that uses a database-independent protocol to communicate database requests to a server component, which then translates the requests into a database-specific protocol. This can simplify deployment since the database-dependent code is located only on the server. It is a three-tier approach of accessing databases. The JDBC clients use standard network sockets to communicate with a middleware application server. The socket information is then translated by the middleware application server into the call format required by the DBMS, and forwarded to the database server. This kind of driver is extremely flexible, since it requires no code installed on the client and a single driver can actually provide access to multiple databases.



We can think of the application server as a JDBC "proxy," meaning that it makes calls for the client application. As a result, we need some knowledge of the application server's configuration in order to effectively use this driver type.

- ❖ A **type 4 driver** is a pure Java library that translates JDBC requests directly to a database-specific protocol. This kind of driver is extremely flexible, we don't need to install special software on the client or server. Further, these drivers can be downloaded dynamically. MySQL's Connector/J driver is an example of Type 4 driver.



Creating JDBC Application:

There are following six steps involved in building a JDBC application:

- Import the packages
- Register the JDBC driver
- Open a connection

- Execute a query
- Extract data from result set
- Clean up the environment

Sample Code:

//STEP 1. Import required packages

```
import java.sql.*;
public class FirstExample {
    // JDBC driver name and database URL
    static final String JDBC_DRIVER = "com.mysql.jdbc.Driver";
    static final String DB_URL = "jdbc:mysql://localhost/EMP";
    // Database credentials
    static final String USER = "username";
    static final String PASS = "password";
    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        try{
            //STEP 2: Register JDBC driver
            Class.forName("com.mysql.jdbc.Driver");
            //STEP 3: Open a connection
            System.out.println("Connecting to database...");
            conn = DriverManager.getConnection(DB_URL,USER,PASS);
            //STEP 4: Execute a query
            System.out.println("Creating statement...");
            stmt = conn.createStatement();
            String sql;
            sql = "SELECT id, first, last, age FROM Employees";
            ResultSet rs = stmt.executeQuery(sql);
            //STEP 5: Extract data from result set
            while(rs.next()){
                //Retrieve by column name
                int id = rs.getInt("id");
                int age = rs.getInt("age");
                String first = rs.getString("first");
                String last = rs.getString("last");
                //Display values
                System.out.print("ID: " + id);
                System.out.print(", Age: " + age);
                System.out.print(", First: " + first);
                System.out.println(", Last: " + last);
            }
            //STEP 6: Clean-up environment
```

```

rs.close();
stmt.close();
conn.close();
}catch(SQLException se){
    //Handle errors for JDBC
    se.printStackTrace();
}catch(Exception e){
    //Handle errors for Class.forName
    e.printStackTrace();
}finally{
    //finally block used to close resources
    try{
        if(stmt!=null)
            stmt.close();
    }catch(SQLException se2){
    }// nothing we can do
    try{
        if(conn!=null)
            conn.close();
    }catch(SQLException se){
        se.printStackTrace();
    }//end finally try
    }//end try
    System.out.println("Goodbye!");
} //end main
} //end FirstExample

```

OUTPUT:

Connecting to database...

Creating statement...

ID: 100, Age: 18, First: Dipendra, Last: Pandey

ID: 101, Age: 25, First: Madan, Last: Pandey

ID: 102, Age: 30, First: Ram, Last: sunar

ID: 103, Age: 28, First: Hari, Last: Pandit

After we've installed the appropriate driver, now we establish a database connection using JDBC. The programming involved to establish a JDBC connection is fairly simple. Here are these simple four steps:

- ❖ **Import JDBC Packages:** Add import statements to our Java program to import required classes in our Java code.
- ❖ **Register JDBC Driver:** This step causes the JVM to load the desired driver implementation into memory so it can fulfill our JDBC requests.
- ❖ **Database URL Formulation:** This is to create a properly formatted address that points to the database to which we wish to connect.

- ❖ **Create Connection Object:** Finally, code a call to the DriverManager object's getConnection() method to establish actual database connection.

1. Creating a Statement for Executing Queries

Connection method **createStatement** is invoked to obtain an object that implements **interface Statement** (package java.sql). The program uses the Statement object to submit SQL statements to the database.

2. Executing a Query

The **Statement** object's **executeQuery** method is used to submit a query that selects all the author information from table Authors. This method returns an object that implements interface **ResultSet** and contains the query results. The ResultSet methods enable the program to manipulate the query result.

3. Processing a Query's ResultSet

The **metadata** describes the ResultSet's contents. Programs can use metadata programmatically to obtain information about the **ResultSet's column names and types**. **ResultSetMetaData** method **getColumnCount** is used to retrieve the number of columns in the ResultSet.

4. Retrieving and Modifying Values from Result Sets

A **ResultSet object** is a table of data representing a database result set, which is usually generated by executing a statement that queries the database. A ResultSet object can be created through any object that implements the Statement interface, including PreparedStatement, CallableStatement, and RowSet.

1. Using Statement Objects for Batch Updates

Statement, PreparedStatement and CallableStatement objects have a list of commands that is associated with them. This list may contain statements for updating, inserting, or deleting a row; and it may also contain DDL statements such as CREATE TABLE and DROP TABLE. It cannot, however, contain a statement that would produce a ResultSet object, such as a SELECT statement. In other words, the list can contain only statements that produce an update count.

The list, which is associated with a Statement object at its creation, is initially empty. We can add SQL commands to this list with the method addBatch and empty it with the method clearBatch. When we have finished adding statements to the list, call the method **executeBatch** to send them all to the database to be executed as a unit, or batch.

```
try {
    connection = DriverManager.getConnection(DATABASE_URL, "root", "");
    connection.setAutoCommit(false);
    statement = connection.createStatement();

    statement.addBatch(
        "INSERT INTO authors " +
        "VALUES('15','Hari','Shrestha')");

    statement.addBatch(
```

```
"INSERT INTO authors " +
"VALUES('16','Ram','Acharya')");
```

```
statement.addBatch(
    "INSERT INTO authors " +
    "VALUES('17','Shyam','Gautam')");
```

```
statement.addBatch(
    "INSERT INTO authors " +
    "VALUES('18','Govinda','Paudel')");
```

```
int [] updateCounts = statement.executeBatch();
connection.commit();
```

```
} catch(BatchUpdateException b) {
    b.printStackTrace();
} catch(SQLException ex) {
    ex.printStackTrace();
}
```

The following line disables auto-commit mode for the Connection object con so that the **transaction** will not be automatically committed or rolled back when the method executeBatch is called.

connection.setAutoCommit(false);

To allow for correct error handling, we should always disable auto-commit mode before beginning a batch update.

The method **Statement.addBatch** adds a command to the list of commands associated with the Statement object statement. In this example, these commands are all INSERT INTO statements, each one adding a row consisting of three column values.

The following line sends the four SQL commands that were added to its list of commands to the database to be executed as a batch:

int [] updateCounts = statement.executeBatch();

Note that **statement** uses the method **executeBatch** to send the batch of insertions, **not the method executeUpdate, which sends only one command and returns a single update count**. The DBMS executes the commands in the order in which they were added to the list of commands, so it will first add the row of values for "Hari", then add the row for "Ram", then "Shyam" , and finally "Govinda". If all four commands execute successfully, the DBMS will return an update count for each command in the order in which it was executed. The update counts that indicate how many rows were affected by each command are stored in the array updateCounts.

If all four of the commands in the batch are executed successfully, updateCounts will contain four values, all of which are 1 because an insertion affects one row. The list of commands associated with stmt will now be empty because the four commands added previously were sent to the database when stmt called the method executeBatch. We can at any time explicitly empty this list of commands with the method clearBatch.

The Connection.commit method makes the batch of updates to the "authors" table permanent. This method needs to be called explicitly because the auto-commit mode for this connection was disabled previously.

The following line enables auto-commit mode for the current Connection object.

```
connection.setAutoCommit(true);
```

Now each statement in the example will automatically be committed after it is executed, and it no longer needs to invoke the method commit.

2. PreparedStatement

A **PreparedStatement** enables us to create **compiled SQL statements** that execute more efficiently than Statements. PreparedStatement can also specify parameters, making them more flexible than Statements—we can execute the same query repeatedly with different parameter values.

The **PreparedStatement** is derived from the more general class, **Statement**. If we want to execute a Statement object many times, it usually reduces execution time to use a PreparedStatement object instead.

Performing Parameterized Batch Update using PreparedStatement

It is also possible to have a parameterized batch update, as shown in the following code fragment, where con is a Connection object:

```
try {
    connection = DriverManager.getConnection(
        DATABASE_URL, "root", "");
    connection.setAutoCommit(false);
    PreparedStatement pstmt = connection.prepareStatement(
        "INSERT INTO authors VALUES(?, ?, ?)");

    pstmt.setInt(1,19);
    pstmt.setString(2, "Navin");
    pstmt.setString(3,"Sharma");
    pstmt.addBatch();

    pstmt.setInt(1,20);
    pstmt.setString(2, "Rajesh");
    pstmt.setString(3,"Paudel");
    pstmt.addBatch();

    // ... and so on for each new
    // type of coffee

    int [] updateCounts = pstmt.executeBatch();
    connection.commit();
}
catch ( SQLException sqlException )
{
    sqlException.printStackTrace();
} // end catch
```

The three question marks (?) in the the preceding SQL statement's last line are placeholders for values

that will be passed as part of the query to the database. Before executing a PreparedStatement, the program must specify the parameter values by using the **PreparedStatement interface's set methods**.

For the preceding query, parameters are int and strings that can be set with PreparedStatement method **setInt and setString**.

Method setInt's and setString's **first argument represents the parameter number being set, and the second argument is that parameter's value**. Parameter numbers are counted from 1, starting with the first question mark (?).

Interface PreparedStatement provides set methods for each supported SQL type. **It's important to use the set method that is appropriate for the parameter's SQL type in the database—SQLExceptions occur when a program attempts to convert a parameter value to an incorrect type.**